INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*JavaScript: Lógica y Manipulación del DOM*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2*
FECHA DE PRESENTACIÓN	*12/10/2026*	**HORA DE PRESENTACIÓN**			
INTEGRANTE (s) *Gomez Huaman Melany Yaritz*				**NOTA (0-20)**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

Demostración de inserción dinámica en el DOM con manejo de eventos y localStorage:

```
// app.js - Módulo de Gestión
const form = document.querySelector('#todo-form');
const input = document.querySelector('#todo-input');
const list = document.querySelector('#todo-list');

let tasks = JSON.parse(localStorage.getItem('tasks')) || [];

function renderTasks() {
    list.innerHTML = '';
    tasks.forEach((task, index) => {
        const li = document.createElement('li');
        li.className = 'list-group-item d-flex justify-content-between align-
items-center';
        li.innerHTML = `
            <span>${task.text}</span>
```

```
                    <button class="btn btn-danger btn-sm"
                onclick="deleteTask(${index})">Eliminar</button>
            `;
            list.appendChild(li);
        });
    }

    form.addEventListener('submit', (e) => {
        e.preventDefault();
        const text = input.value.trim();
        if (!text) return;

        tasks.push({ text, completed: false });
        localStorage.setItem('tasks', JSON.stringify(tasks));
        input.value = '';
        renderTasks();
    });

    function deleteTask(index) {
        tasks.splice(index, 1);
        localStorage.setItem('tasks', JSON.stringify(tasks));
        renderTasks();
    }

    document.addEventListener('DOMContentLoaded', renderTasks);
```

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de Tareas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css
" rel="stylesheet">
</head>
<body class="container py-4">
  <h1 class="mb-4">Lista de Tareas</h1>

  <form id="todo-form" class="d-flex gap-2 mb-3">
    <input type="text" id="todo-input" class="form-control"
placeholder="Nueva tarea...">
    <button type="submit" class="btn btn-primary">Agregar</button>
  </form>
```

```
  <ul id="todo-list" class="list-group"></ul>

  <script src="app.js"></script>
</body>
</html>
```

app-js

```
const form = document.querySelector('#todo-form');
const input = document.querySelector('#todo-input');
const list = document.querySelector('#todo-list');

let tasks = JSON.parse(localStorage.getItem('tasks')) || [];

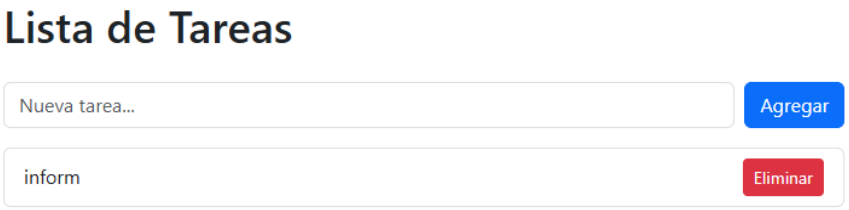
function renderTasks() {
    list.innerHTML = '';
    tasks.forEach((task, index) => {
        const li = document.createElement('li');
        li.className = 'list-group-item d-flex justify-content-between
align-items-center';
        li.innerHTML = `
            <span>${task.text}</span>
            <button class="btn btn-danger btn-sm"
onclick="deleteTask(${index})">Eliminar</button>
        `;
        list.appendChild(li);
    });
}

form.addEventListener('submit', (e) => {
    e.preventDefault();
    const text = input.value.trim();
    if (!text) return;

    tasks.push({ text, completed: false });
    localStorage.setItem('tasks', JSON.stringify(tasks));
    input.value = '';
    renderTasks();
});
```

```
function deleteTask(index) {
    tasks.splice(index, 1);
    localStorage.setItem('tasks', JSON.stringify(tasks));
    renderTasks();
}

document.addEventListener('DOMContentLoaded', renderTasks);
```



II. PROBLEMAS PROPUESTOS RESUELTOS:

Desarrollar una aplicación interactiva "Gestor de Tareas Universitarias" en el navegador cliente sin refresco de página:

1. Gestión del Estado y Estructuras de Datos:

○ Mantener un arreglo de objetos JavaScript global para las tareas: [{ id, titulo, curso, fechaEntrega, completada }].

○ Utilizar métodos iterativos ES6+ (map, filter, reduce, find) para la manipulación del arreglo.

2. Captura y Validación con JavaScript:

○ Interceptar el evento submit del formulario de alta mediante preventDefault().

○ Validar mediante JS que ningún campo esté vacío y que la fecha de entrega sea posterior a la fecha actual. En caso de error, mostrar mensajes dinámicos en un div de alertas.

3. Manipulación Dinámica del DOM:

○ Renderizar la lista de tareas dinámicamente construyendo elementos DOM o usando template literals seguros.

○ Permitir alternar el estado de la tarea (completada/pendiente), modificando sus clases CSS dinámicamente (ej. tachado de texto).

○ Permitir la eliminación individual de una tarea del DOM y del arreglo.

○ Implementar un filtro visual (Todas / Pendientes / Completadas) manipulando el DOM.

4. Persistencia de Datos:

○ Sincronizar cada adición, cambio de estado o eliminación con localStorage.

○ Cargar y renderizar las tareas guardadas automáticamente al desencadenarse el evento DOMContentLoaded.

index.html

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Gestor de Tareas Universitarias</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

  <link href="styles.css" rel="stylesheet">

</head>

<body>

  <main class="container py-4" style="max-width: 800px;">

    <h1 class="mb-4">Gestor de Tareas Universitarias</h1>


    <div id="alertas"></div>


    <form id="task-form" class="card card-body mb-4" novalidate>

      <div class="row g-3">

        <div class="col-md-12">

          <label for="titulo" class="form-label">Título</label>
```

```
          <input type="text" id="titulo" class="form-control" placeholder="Ej. Informe de laboratorio 04">
        </div>
        <div class="col-md-6">
          <label for="curso" class="form-label">Curso</label>
          <input type="text" id="curso" class="form-control" placeholder="Ej. Introducción al Desarrollo Web">
        </div>
        <div class="col-md-6">
          <label for="fechaEntrega" class="form-label">Fecha de entrega</label>
          <input type="date" id="fechaEntrega" class="form-control">
        </div>
        <div class="col-12">
          <button type="submit" class="btn btn-primary">Agregar tarea</button>
        </div>
      </div>
    </form>


    <div class="d-flex justify-content-between align-items-center mb-3 flex-wrap gap-2">
      <div class="btn-group" id="filtros" role="group">
        <button type="button" class="btn btn-outline-secondary active" data-filter="todas">Todas</button>
        <button type="button" class="btn btn-outline-secondary" data-filter="pendientes">Pendientes</button>
        <button type="button" class="btn btn-outline-secondary" data-filter="completadas">Completadas</button>
```

```
      </div>
      <small id="resumen" class="text-muted"></small>
    </div>

    <ul id="task-list" class="list-group"></ul>
  </main>

  <script src="app.js"></script>
</body>
</html>
```

styles.css

```
.tarea-completada .tarea-titulo {
  text-decoration: line-through;
  color: #92bee6;
}
.tarea-completada {
  background-color: #f8f9fa;
}
.tarea-vencida .tarea-fecha {
  color: #dc3545 !important;
  font-weight: 600;
}
```

app.js

```
//  PASO 1: estado global y persistencia
// Estructura de cada tarea: { id, titulo, curso, fechaEntrega, completada }

const STORAGE_KEY = 'tareasUniversitarias';

// Lee las tareas guardadas. Si no hay nada o el JSON está dañado, devuelve []
function cargarTareas() {
  try {
    const guardadas = JSON.parse(localStorage.getItem(STORAGE_KEY));
    return Array.isArray(guardadas) ? guardadas : [];
  } catch (error) {
    console.error('No se pudo leer localStorage:', error);
    return [];
  }
}

// Estado inicial: lo que haya guardado
let tareas = cargarTareas();

// Guarda el arreglo completo como texto JSON
const guardarTareas = () => {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tareas));
};
```

```
// ---------- Funciones sobre el estado ----------

// Agregar (destructuring + spread)
const agregarTarea = ({ titulo, curso, fechaEntrega }) => {
  const nueva = { id: Date.now(), titulo, curso, fechaEntrega, completada:
false };
  tareas = [...tareas, nueva];
  guardarTareas();
};

// find: buscar por id
const buscarTarea = (id) => tareas.find((t) => t.id === id);

// map: alternar completada/pendiente
const alternarTarea = (id) => {
  tareas = tareas.map((t) => (t.id === id ? { ...t, completada: !t.completada }
: t));
  guardarTareas();
};

// filter: eliminar
const eliminarTarea = (id) => {
  tareas = tareas.filter((t) => t.id !== id);
  guardarTareas();
};
```

```
// filter: filtrar por estado
const filtrarTareas = (filtro) => {
  if (filtro === 'pendientes') return tareas.filter((t) => !t.completada);
  if (filtro === 'completadas') return tareas.filter((t) => t.completada);
  return tareas;
};

// reduce: contar
const contarTareas = () =>
  tareas.reduce(
    (acc, t) => ({
      total: acc.total + 1,
      completadas: acc.completadas + (t.completada ? 1 : 0),
      pendientes: acc.pendientes + (t.completada ? 0 : 1),
    }),
    { total: 0, completadas: 0, pendientes: 0 }
  );



//PASO 2: CAPTURA Y VALIDACIÓN

// Referencias al DOM
const form = document.querySelector('#task-form');
const inputTitulo = document.querySelector('#titulo');
```

```
const inputCurso = document.querySelector('#curso');
const inputFecha = document.querySelector('#fechaEntrega');
const contenedorAlertas = document.querySelector('#alertas');


// Fecha de hoy en formato AAAA-MM-DD (hora local, no UTC)
const fechaHoy = () => {
  const d = new Date();
  const mes = String(d.getMonth() + 1).padStart(2, '0');
  const dia = String(d.getDate()).padStart(2, '0');
  return `${d.getFullYear()}-${mes}-${dia}`;
};


//  Validación


// Devuelve un arreglo de errores (vacío = todo válido)
const validarTarea = ({ titulo, curso, fechaEntrega }) => {
  const errores = [];


  if (!titulo) errores.push('El título es obligatorio.');
  if (!curso) errores.push('El curso es obligatorio.');


  if (!fechaEntrega) {
    errores.push('La fecha de entrega es obligatoria.');
  } else if (fechaEntrega <= fechaHoy()) {
    errores.push('La fecha de entrega debe ser posterior a la fecha actual.');
```

```
  }


  return errores;
};


// Muestra mensajes en el div de alertas.
// textContent (no innerHTML) evita inyección de HTML.
const mostrarAlertas = (mensajes, tipo = 'danger') => {
  contenedorAlertas.innerHTML = '';
  mensajes.forEach((mensaje) => {
    const div = document.createElement('div');
    div.className = `alert alert-${tipo} py-2 mb-2`;
    div.setAttribute('role', 'alert');
    div.textContent = mensaje;
    contenedorAlertas.appendChild(div);
  });
};


// Evento submit
form.addEventListener('submit', (e) => {
  e.preventDefault(); // evita que la página se recargue


  const datos = {
    titulo: inputTitulo.value.trim(),
    curso: inputCurso.value.trim(),
```

```
    fechaEntrega: inputFecha.value,
  };

  const errores = validarTarea(datos);

  if (errores.length > 0) {
    mostrarAlertas(errores);
    return; // se detiene: no se agrega nada
  }

  agregarTarea(datos);
  form.reset();
  inputTitulo.focus();
  mostrarAlertas(['Tarea agregada correctamente.'], 'success');
  renderizar();
});



//  PASO 3: RENDERIZADO DINÁMICO

//  Referencias al DOM
const lista = document.querySelector('#task-list');
const resumen = document.querySelector('#resumen');

// Filtro activo
```

```
let filtroActual = 'todas';


// Construye el <li> de UNA tarea
const crearElementoTarea = ({ id, titulo, curso, fechaEntrega, completada }) =>
{
  const li = document.createElement('li');
  li.dataset.id = id; // guarda el id en el elemento
  li.className = 'list-group-item d-flex justify-content-between
align-items-center gap-3';


  // Clases CSS según el estado
  const vencida = !completada && fechaEntrega <= fechaHoy();
  if (completada) li.classList.add('tarea-completada');
  if (vencida) li.classList.add('tarea-vencida');


  // Checkbox (alterna el estado)
  const check = document.createElement('input');
  check.type = 'checkbox';
  check.className = 'form-check-input mt-0';
  check.checked = completada;
  check.dataset.action = 'toggle';


  // Bloque de texto: título + detalle
  const info = document.createElement('div');
  info.className = 'flex-grow-1';
```

```
  const tituloEl = document.createElement('div');
  tituloEl.className = 'tarea-titulo fw-semibold';
  tituloEl.textContent = titulo;


  const detalleEl = document.createElement('small');
  detalleEl.className = 'text-muted tarea-fecha';
  detalleEl.textContent = `${curso} · Entrega: ${fechaEntrega}${vencida ? '
(vencida)' : ''}`;


  info.append(tituloEl, detalleEl);


  // Botón eliminar
  const btnEliminar = document.createElement('button');
  btnEliminar.type = 'button';
  btnEliminar.className = 'btn btn-danger btn-sm';
  btnEliminar.dataset.action = 'delete';
  btnEliminar.textContent = 'Eliminar';


  li.append(check, info, btnEliminar);
  return li;
};


// Dibuja toda la lista según el filtro actual
const renderizar = () => {
  lista.innerHTML = ''; // limpia SIEMPRE antes de dibujar (evita duplicados)
```

```
  const visibles = filtrarTareas(filtroActual);

  if (visibles.length === 0) {
    const vacio = document.createElement('li');
    vacio.className = 'list-group-item text-muted text-center';
    vacio.textContent = 'No hay tareas para mostrar.';
    lista.appendChild(vacio);
  } else {
    visibles.map(crearElementoTarea).forEach((li) => lista.appendChild(li));
  }

  // Resumen con reduce
  const { total, completadas, pendientes } = contarTareas();
  resumen.textContent = `${total} en total · ${pendientes} pendientes · ${completadas} completadas`;
};

// Carga inicial: dibuja las tareas guardadas
document.addEventListener('DOMContentLoaded', renderizar);



// PASO 4: INTERACTIVIDAD (DELEGACIÓN DE EVENTOS)

const filtros = document.querySelector('#filtros');

// Un solo listener para TODOS los checkbox y botones Eliminar de la lista
```

```
lista.addEventListener('click', (e) => {
  // closest busca el elemento con data-action más cercano al clic
  const elemento = e.target.closest('[data-action]');
  if (!elemento) return; // el clic fue en otra parte del <li>: se ignora

  // Leemos el id guardado en el <li> padre (data-id)
  const id = Number(elemento.closest('li').dataset.id);

  if (!buscarTarea(id)) return; // por seguridad

  if (elemento.dataset.action === 'toggle') alternarTarea(id);
  if (elemento.dataset.action === 'delete') eliminarTarea(id);

  renderizar();
});

// Delegación también para los botones de filtro
filtros.addEventListener('click', (e) => {
  const boton = e.target.closest('[data-filter]');
  if (!boton) return;

  filtroActual = boton.dataset.filter;

  // Marca como activo solo el botón presionado
  filtros.querySelectorAll('button').forEach((b) => {
```

```
    b.classList.toggle('active', b === boton);
  });


  renderizar();
});
```



Repositorio Github: https://github.com/mgomezhua-creator/IDWeb/tree/main/Lab04

III. CUESTIONARIO:

1. ¿Cuál es la diferencia de comportamiento entre las palabras clave var, let y const respecto al alcance (scope) y al hoisting?

Respecto al alcance, var tiene alcance de función. Ignora los bloques de if, for, while, la variable sigue existiendo fuera de ellos. Let y const tienen alcance de bloque. Solo existen dentro de las llaves {} donde se declaran.

Respecto al hoisting, var es elevada a la parte superior de su función o ámbito global y se inicializa automáticamente con el valor undefined ; let y const también sufren *hoisting*, pero no se inicializan.

Entran en una zona llamada TDZ (*Temporal Dead Zone* o Zona Muerta Temporal) que va desde el principio del bloque hasta que la línea de código donde se declara la variable es ejecutada.

2. Explique la técnica de Delegación de Eventos (Event Delegation) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.

La delegación de eventos es una técnica en la que, en lugar de poner un listener en cada elemento hijo, se pone un solo listener en un elemento padre y desde ahí se averigua qué hijo originó el evento. Funciona gracias al burbujeo (bubbling): cuando ocurre un evento en un elemento, como un clic en un botón, el evento no se queda ahí, sino que sube por el árbol del DOM pasando por sus ancestros (button → li → ul → body → document). El padre puede "escucharlo" en su camino hacia arriba y usar event.target para saber quién lo originó.

Es eficiente porque funciona con elementos dinámicos, abarca menos memoria y menos trabajo en el navegador y mantiene el código limpio.

3. ¿Por qué es necesario utilizar JSON.stringify() al almacenar objetos complejos en localStorage y qué limitaciones tiene este almacenamiento en términos de seguridad?

Es necesario utilizar JSON.stringify() porque localStorage solo admite cadenas de texto (strings) como valores.El objeto se serializa en una cadena con formato JSON, preservando toda su estructura para que luego pueda ser recuperada intacta mediante JSON.parse().

Sin embargo, tiene limitaciones en cuanto a seguridad como vulnerabilidad a ataques XXS, falta de cifrado, ausencia de restricciones de lectura y alcance por origen.

CONCLUSIONES

El laboratorio me permitió utilizar JavaScript para construir una aplicación interactiva con una interfaz dinámica, uso del DOM y eventos con validación de datos. Del mismo modo podría utilizar más seguido los métodos iterativos de ES6+ ya que reemplazan los bucles tradicionales con código más claro, por último podría mejorar en el orden del código ya que a largo plazo si es importante.

METODOLOGÍA DE TRABAJO

1. Análisis del problema y lectura de la guía
2. Ejecución del ejercicio resuelto por el docente
3. Preparación de la estructura base
4. Implementación por etapas en `app.js`
4.1 Estado y estructuras de datos
4.2 Captura y validación

4.3 Renderizado del DOM
4.4 Interactividad
4.5 Persistencia
5. 5. Pruebas y depuración
6. Revisión contra la rúbrica y desarrollo del cuestionario y conclusiones

REFERENCIAS Y BIBLIOGRAFÍA

- *Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.*
- *MDN Web Docs. (2025). Document Object Model (DOM) & Events. Developer Mozilla.*